

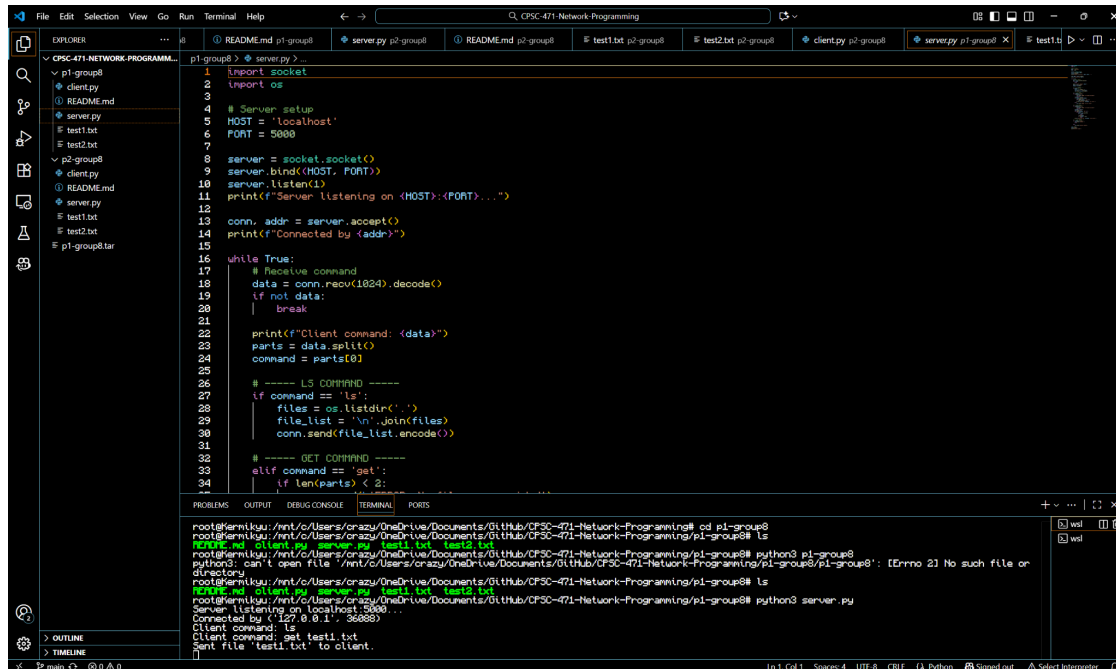
CPSC 471 Socket Programming Project

Phase 1 Progress Report

Bryan Tran | btran299@csu.fullerton.edu
Peter Chau | Peterchau93@csu.fullerton.edu

Current Status of Phase 1:

server.py

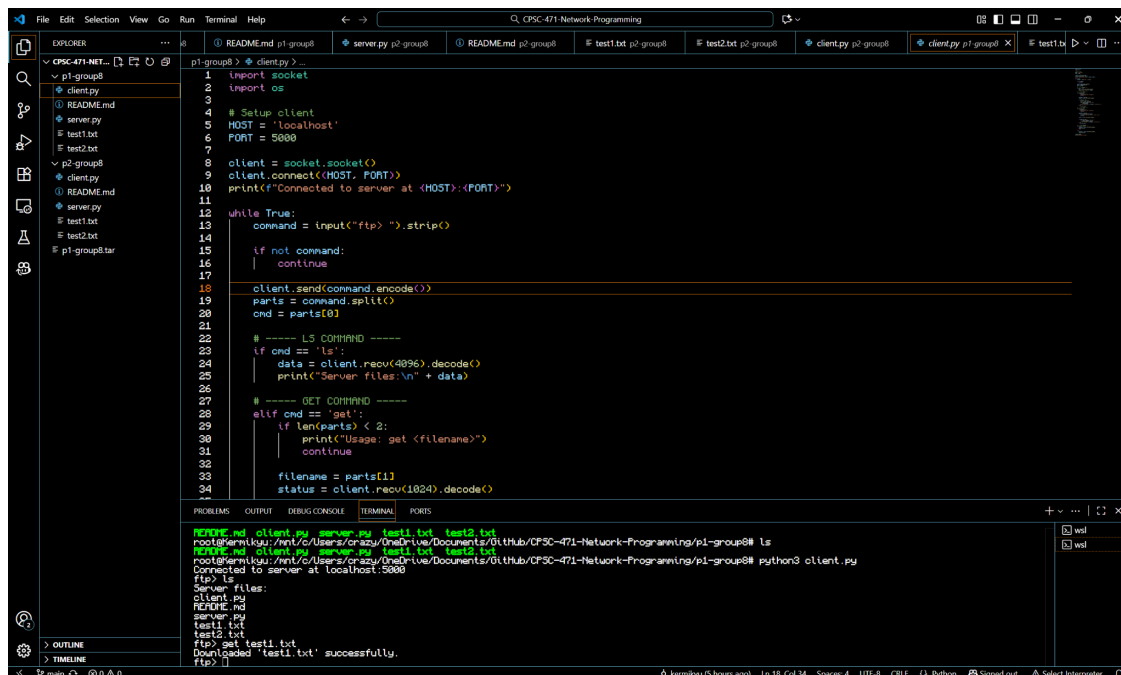


```
1 import socket
2 import os
3
4 # Server setup
5 HOST = 'localhost'
6 PORT = 5000
7
8 server = socket.socket()
9 server.bind((HOST, PORT))
10 server.listen(1)
11 print(f"Server listening on {HOST}:{PORT}...")
12
13 conn, addr = server.accept()
14 print(f"Connected by {addr}")
15
16 while True:
17     # Receive command
18     data = conn.recv(1024).decode()
19     if not data:
20         break
21
22     print(f"Client command: {data}")
23     parts = data.split()
24     command = parts[0]
25
26     # ---- LS COMMAND ----
27     if command == 'ls':
28         files = os.listdir('.')
29         file_list = '\n'.join(files)
30         conn.send(file_list.encode())
31
32     # ---- GET COMMAND ----
33     elif command == 'get':
34         if len(parts) < 2:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
root@kali:~/CSC-471-Network-Programming# cd p1-group8
root@kali:~/CSC-471-Network-Programming/p1-group8# ls
README.md client.py server.py test1.txt test2.txt
root@kali:~/CSC-471-Network-Programming/p1-group8# python3 p1-group8
python3: can't open file '/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CSC-471-Network-Programming/p1-group8/p1-group8': [Errno 2] No such file or directory
root@kali:~/CSC-471-Network-Programming/p1-group8# ls
README.md client.py server.py test1.txt test2.txt
root@kali:~/CSC-471-Network-Programming/p1-group8# python3 server.py
Server listening on localhost:5000...
Connected by ('127.0.0.1', 36088)
Client command: ls
Client command: get test1.txt
Sent file 'test1.txt' to client.
```

client.py



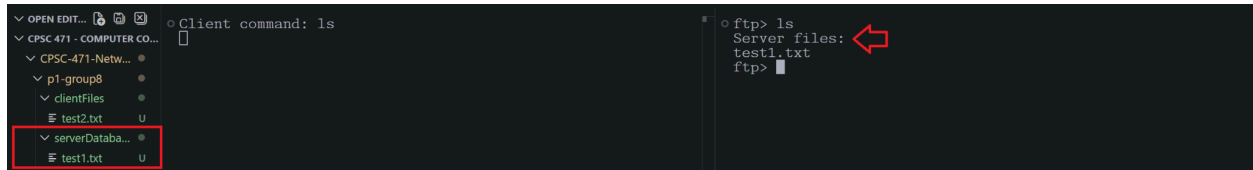
```
1 import socket
2 import os
3
4 # Setup client
5 HOST = 'localhost'
6 PORT = 5000
7
8 client = socket.socket()
9 client.connect((HOST, PORT))
10 print(f"Connected to server at {HOST}:{PORT}")
11
12 while True:
13     command = input("ftp> ").strip()
14     if not command:
15         continue
16
17     client.send(command.encode())
18     parts = command.split()
19     cmd = parts[0]
20
21     # ---- LS COMMAND ----
22     if cmd == 'ls':
23         data = client.recv(4096).decode()
24         print("Server files:\n" + data)
25
26     # ---- GET COMMAND ----
27     elif cmd == 'get':
28         if len(parts) < 2:
29             print("Usage: get <filename>")
30             continue
31
32         filename = parts[1]
33         status = client.recv(1024).decode()
34         print(status)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
root@kali:~/CSC-471-Network-Programming/p1-group8# ls
README.md client.py server.py test1.txt test2.txt
root@kali:~/CSC-471-Network-Programming/p1-group8# python3 client.py
Connected to server at localhost:5000
ftp> ls
Server files:
client.py
README.md
server.py
test1.txt
test2.txt
ftp> get test1.txt
Downloaded 'test1.txt' successfully.
ftp>
```

Screenshots provided separate server and client response (left - server | right - client)

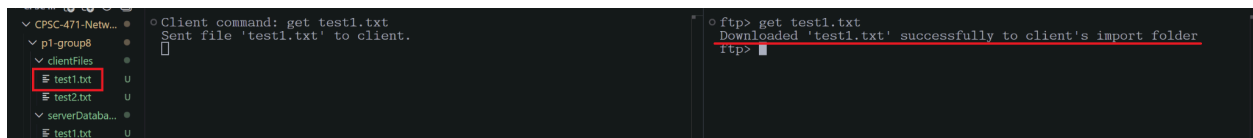
LS command:



```
Client command: ls
ftp> ls
Server files:
test1.txt
ftp>
```

- Client's visibility only within the server's database, displaying all existing files within the 'serverDatabase' directory

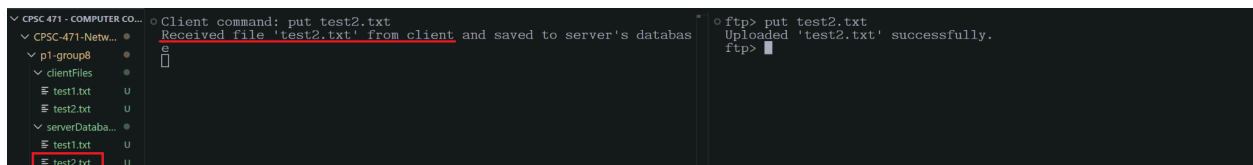
GET command:



```
Client command: get test1.txt
Sent file 'test1.txt' to client.
ftp> get test1.txt
Downloaded 'test1.txt' successfully to client's import folder
ftp>
```

- Client is able to grab 'test1.txt' from the server's database and download it into the client's 'clientFiles' folder

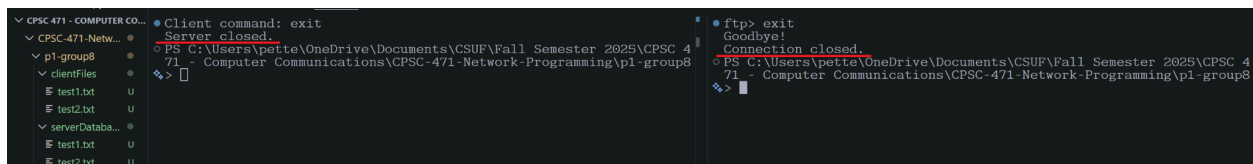
PUT command:



```
Client command: put test2.txt
Received file 'test2.txt' from client and saved to server's databas
Server closed.
ftp> put test2.txt
Uploaded 'test2.txt' successfully.
ftp>
```

- Client is able to send its own file, 'test2.txt', across the socket connection, which is then verified by the server and then stored in the server's database

EXIT command:



```
Client command: exit
Server closed.
ftp> exit
Goodbye!
```

- Once the client terminates the connection, the server subsequently sends a final 'Goodbye!' message and closes its connection alongside the client

Future outlook for Phase 2:

Both teammates share equal responsibility for the project by reviewing each other's code and testing all functionality to ensure everything works correctly and consistently. Future incorporations will involve multiprocessing capabilities on the server side to handle multiple client communication lines. Clients should still retain the ability to perform the appropriate commands established in phase 1. Incorporate a database architecture that allows for logging/monitoring of network traffic information on some storage device. Test each functionality separately and evaluate performance to resolve conflicts or communication issues before deploying onto the cloud through AWS buckets for further concurrency assessment.

Timeline:

November 15: Bryan and Peter will expand on client functionality for multi-concurrency, simulating tests with multiple clients.

November 19: Determine which method of logging is most preferred and begin implementing the database structure, aggregating data locally whenever the server is active.

November 23: Begin deployment into the AWS cloud, resolving possible run-ins with subscription and instantiating the server as well as client connections from multiple sources (local or on cloud).

November 27: Conclude with further testing and monitoring, verifying functionality and availability submission on November 30th.